Salibot

Complete Application Guide

How the website, WhatsApp bots, Railway hosting, data, security, and deployment work

Written in plain language for operators and non-technical readers

Application type:	Multi-bot WhatsApp control server
Hosting target:	Railway
Main website language:	French
Guide date:	10 September 2026

Contents

1 The application in one minute

Salibot is one application with two faces:

- a private administration website where an operator creates, connects, configures, and watches WhatsApp bot instances; and
- a background WhatsApp worker that listens for events and commands, sends alerts, recovers certain deleted messages, forwards statuses, and downloads supported social videos.

Railway supplies the internet-facing server. GitHub stores the source code and runs the pre-deployment security checks. Cloudflare Turnstile can add a human-verification challenge to the login page. The application uses the Baileys library to behave like a linked WhatsApp device.

In simple terms: A person opens the website, signs in, creates a bot, links a WhatsApp account, chooses which groups matter, and turns features on or off. Salibot then keeps running on Railway and performs those choices automatically.

Important: This is not the official WhatsApp Business Cloud API. It uses a linked-device session. WhatsApp can disconnect a session, change behavior, or require the account to be linked again. The operator is responsible for lawful use, consent, privacy, and compliance with WhatsApp's terms.

1.1 What the application does not do

The current application does not provide an unrestricted command-line shell inside the website. The page called **Console serveur** is a read-only view of application logs. It cannot run Linux commands. Railway build logs and Railway platform events stay in Railway's own dashboard.

2 The big picture

2.1 Main parts

Part	Plain-language purpose
Railway service	Keeps the application online, provides a public address, restarts it after failures, and can attach persistent storage.
Web administration site	The private control panel used to sign in, manage bots, change features, view resources, and read logs.
WhatsApp bot engine	Opens one linked WhatsApp connection for every configured bot and reacts to WhatsApp events.
Data folder	Stores the bot list, settings, website password record, WhatsApp linked-device credentials, cached media, and logs.
External services	WhatsApp, Cloudflare Turnstile, AI providers, social-media sites, optional Cobalt, and the Cloudflare speed-test endpoint.
GitHub security pipeline	Runs thirteen checks before a release is allowed to deploy.

2.2 A normal request from start to finish

Website flow
Browser ⟶ Railway public address ⟶ Salibot login ⟶ protected dashboard ⟶ saved settings or bot action

WhatsApp flow
WhatsApp event ⟶ linked bot receives it ⟶ Salibot checks the bot's rules ⟶ Salibot ignores it or sends the chosen alert/action

Deployment flow
Code pushed to GitHub ⟶ thirteen security stages ⟶ final gate passes ⟶ Railway deploys the same revision

2.3 What happens when the server starts

1. Node.js starts `index.js` in the production container.
2. Console messages begin going to both Railway's log stream and `logs/server.log`.
3. The application chooses its data directory. If Railway attached a volume, the Railway mount path is used automatically; otherwise files live beside the application.
4. It loads `bots.json`. If that file does not exist, it creates one bot named `bot1`.
5. It starts the protected web administration site.
6. It schedules cleanup of old recovered-media cache files and starts RAM/disk monitoring.
7. Unless WhatsApp was explicitly disabled, it starts every configured bot.
8. Each bot loads its saved linked-device credentials and attempts to connect to WhatsApp.

3 Website tour

3.1 Login page

The login page asks for the administrator name and password. On the first start, these come from `WEB_ADMIN_USER` and `WEB_ADMIN_PASSWORD`. The password is converted into a one-way password record before being saved; the original password is not stored in `auth.json`.

When both `TURNSTILE_SITE_KEY` and `TURNSTILE_SECRET_KEY` exist, the Cloudflare Turnstile widget appears and the server verifies its token before checking the password. If either Turnstile value is absent, the current code disables Turnstile completely. For the intended protected production setup, configure both.

After a successful login, the browser receives a private session cookie. The session lasts twelve hours from the most recent activity. Sessions are held in memory, so a server restart signs everyone out.

3.2 Main dashboard: Serveur Multi-Bots

The main dashboard gives the operator a single view of the whole Railway instance:

- whether WhatsApp processing is active;
- every bot name and whether it is connected;
- buttons to create and manage bots;
- host and container RAM estimates;
- host and application-disk estimates;
- application uptime;
- a download-speed and latency test; and
- a button to restart the whole application.

The RAM and disk figures are operational estimates. The container calculation assumes 512 MB of RAM and 1 GB of application disk. If the Railway plan uses different limits, the percentages are not billing figures and may not match the Railway dashboard exactly.

The speed test downloads up to about 10 MB from Cloudflare's speed endpoint. It consumes network bandwidth and normally takes several seconds.

The restart button exits the application process. Railway sees that exit and starts a replacement according to `railway.json`. During the restart, website sessions are lost and WhatsApp bots reconnect.

3.3 Creating a bot

Enter a bot name such as `bot_support` and select **Créer un bot**. Bot names are cleaned so only letters, numbers, hyphens, and underscores are used in file paths. The bot is added to `bots.json` and starts automatically when WhatsApp processing is active.

Each bot is independent. It has its own WhatsApp linked-device folder, allowed groups, tracked groups, feature choices, connection state, and temporary message memory.

3.4 Connecting a bot to WhatsApp

There are two methods:

1. **QR code.** Open the bot, select **Connecter par QR Code**, then scan it from WhatsApp under Linked devices / Link a device. If the QR is not ready, wait a few seconds and refresh.
2. **Phone number.** Select **Connecter par Numéro**, enter the full number with country code and digits only, request a linking code, then enter that code in WhatsApp's linked-device flow.

When connection succeeds, the website changes to **Connecté**, the QR and pairing code are cleared, the group list is refreshed, and queued server alerts are sent. A temporary network disconnect triggers an automatic reconnect. If WhatsApp reports that the account was logged out, Salibot clears that bot's saved session and a new link is required.

3.5 Disconnecting and deleting

Disconnect logs the WhatsApp device out but keeps the bot entry and its feature/group settings. The bot can be linked again later.

Delete bot is more permanent. It removes the bot from the bot list, logs it out, deletes the linked-device credential folder, and deletes its allowed/tracked-group settings file. Use it only when the bot is no longer needed.

3.6 Allowed groups and tracked groups

These two lists have different jobs:

List	Meaning
Allowed groups	Normally the destinations that receive alerts, forwarded statuses, recovered-message reports, and videos sent by the `.u` command.
Tracked groups	The source groups Salibot watches for deleted messages, member changes, administrator changes, and group-setting changes.

In simple terms: A tracked group is where Salibot watches. An allowed group is where Salibot sends. A group may be in both lists, one list, or neither.

The group selection pages can only list groups while the bot is connected and is already a member of those groups.

3.7 Per-bot permissions

The setting **Who can use commands?** has three choices:

- **Everyone**: any participant may use ordinary commands where the location rule allows them.
- **Group administrators**: the linked account and administrators in group chats may use ordinary commands.
- **Owner only**: only messages sent by the linked account itself may use commands.

The setting **Where can commands be used?** has four choices: everywhere, private chats only, allowed groups only, or private chats plus allowed groups.

The setting **Where should alerts go?** chooses either the allowed groups or a private message to the linked account owner.

Regardless of the general choice, `.bots` and `.reboot` are always owner-only. The social download command `.u` is also owner-only.

3.8 Feature switches

New bots start with all feature switches enabled. Each switch changes only the selected bot.

Feature shown on the website	What it does
Download Reels with `.u`	Allows the owner to download a supported social-media video and send it to all allowed groups.
Anti-delete: private messages	Remembers incoming private messages so a deletion can produce a report and, when possible, a recovered copy.
Anti-delete: tracked groups	Does the same for messages inside groups selected as tracked.
Forward new statuses	Forwards new WhatsApp status text, images, or videos to the chosen alert destinations.
Group member notifications	Reports joins/additions and leaves/removals in tracked groups.
Administrator notifications	Reports promotions and demotions in tracked groups.
Group-setting notifications	Reports tracked-group name, picture, description, editing-rule, and messaging-rule changes.
Contact-photo notifications	Reports when a contact appears to change a profile picture; the bot's own picture is excluded.
Capture View Once: private	Attempts to preserve and forward incoming view-once private content.
Capture View Once: groups	Attempts to preserve and forward incoming view-once group content. This switch is not limited to the tracked-group list in the current code.

Select **Save settings** after changing switches or permission menus.

3.9 Console serveur: the log viewer

The sidebar item **Logs serveur** opens `/logs`. It shows the recent end of `logs/server.log`, refreshes every two seconds, scrolls with new output when the operator is already near the bottom, and has a pause/resume control. The operator may display 200, 500, 1,000, or 2,000 lines. Each read is limited to the newest 512 KB so the page does not load an unlimited file.

The log page and its `/api/logs` feed require an authenticated administrator session. Dynamic content is placed on the page as text, not executable HTML. Successful health checks and successful log polling are not logged, which prevents the viewer from filling its own output.

The file contains application runtime output such as connection changes, commands, errors, downloads, and web requests. It does not include Docker build output or every Railway platform event. It is not an interactive terminal and cannot execute commands.

4 What happens inside WhatsApp

4.1 How Salibot listens

For every configured bot, Salibot creates a Baileys socket using that bot's saved linked-device credentials. It subscribes to connection changes, credential updates, message changes, new messages, group participant updates, and contact updates.

Salibot ignores many events unless their matching feature is enabled. It also checks whether a message is private or in a group, whether the group is tracked or allowed, who sent the command, and where alerts should go.

4.2 Anti-delete behavior

For an incoming private message, Salibot remembers the message in memory when private anti-delete is enabled. For a group message, it remembers the message only when the group is tracked. Supported media is also downloaded into the deleted-media cache when possible.

If WhatsApp later reports a delete/revoke event, Salibot looks up the original message. It sends a report containing the sender, chat or group, time, media type, and text, followed by the recovered media or forwarded message when possible. After processing the deletion, the in-memory entry is removed.

Media types include images, normal videos, video notes, audio or voice notes, stickers, and documents. Old files in the deleted-media cache are removed after seven days by default. Cleanup starts shortly after boot and then runs every six hours by default.

Important: Recovery is best effort, not a guarantee. Salibot must have been online and must have received the original message before deletion. WhatsApp encryption or linked-device behavior may prevent access, especially for View Once content. Text/message memory is lost on restart even when media files remain on disk.

4.3 Group and contact notifications

Tracked-group participant events can report additions, removals, promotions, and demotions. Tracked-group system messages can report changes to the group name, picture, description, editing permissions, or message permissions. Contact update events can report profile-picture changes.

These reports go to the configured alert destination. If the destination is **allowed groups** but no allowed group exists, some alert types have nowhere to go. System resource alerts have an extra fallback to the linked account owner.

4.4 Status and View Once forwarding

When status forwarding is enabled, incoming status text is included in a report. Images and videos are downloaded and sent when available. View Once forwarding attempts to extract the original content and send it with a sender/group notice. When WhatsApp blocks the content, Salibot sends a notice explaining that the media was unavailable.

Important: These features can copy content that another person expected to disappear or remain limited. Use them only where everyone involved has been informed and where local law permits it. Protect destinations, backups, and logs from unauthorized access.

4.5 Resource alerts

About twenty seconds after startup, and every five minutes afterward, Salibot checks its estimated container RAM and disk use. The default warning level is 85%. Repeated high alerts are limited by a thirty-minute cooldown. When usage returns below the threshold, Salibot sends a recovery message.

If no bot is connected, an alert can be queued for up to one hour. The queue holds at most fifty entries and tries again after a bot reconnects. Duplicate alerts with the same first line are updated rather than endlessly added.

5 WhatsApp command reference

Commands begin with a period by default. For example, the user sends `.ping`. The advanced `COMMAND_PREFIX` setting can change the prefix.

Command	Who	Result
`.ping`	According to permissions	Replies `pong` to prove the bot is responding.
`.stats`	According to permissions	Shows estimated container RAM and application-disk use.
`.servers`	According to permissions	Shows this standalone Railway server, its WhatsApp mode, uptime, and connected-bot count.
`.uptime`	According to permissions	Shows how long the current application process has been running.
`.test`	According to permissions	Runs the server download-speed and latency test and replies with the result.
`.ask question`	According to permissions; AI must be configured	Sends one question to the selected AI provider and returns its answer.
`.u URL`	Owner only	Downloads supported social video links and sends successful results to every allowed group.
`.bots`	Owner only	Starts an interactive menu to list, create, delete, connect, or disconnect bots.
`.reboot`	Owner only	Exits the server process so Railway can restart it.
`.help`	According to permissions	Displays command explanations.
`.menu`	According to permissions	Displays the short command menu.
`.exit`	Flow owner	Cancels an interactive bot-management flow. During a flow, `exit`, `quitter`, `annuler`, or `retour` also cancels.

The bot-management conversation remembers who started it. In a group, another participant cannot take over the original person's menu flow.

6 Social-video downloading

The `.u` command recognizes Instagram, TikTok, Facebook, YouTube, and X/Twitter URLs. It removes common tracking parameters and duplicate links before processing.

The normal download engine is `yt-dlp`, supplied by `youtube-dl-exec`. It prefers video up to 720p and MP4-compatible formats so the result is more suitable for WhatsApp. A download normally times out after ninety seconds and is limited to roughly 80 MB. Temporary files are deleted after sending.

An operator may optionally configure one or more Cobalt endpoints. When configured, Cobalt is tried first except for Instagram stories; `yt-dlp` remains the fallback. Remote results can be downloaded to a temporary local file before they are sent, which helps validate size and avoid expired URLs.

Private, deleted, expired, age-restricted, geo-blocked, or login-protected media may fail. Instagram stories commonly need cookies. A proxy or custom cookie source is an advanced operator option and must be protected like a password.

7 AI assistant

The `.ask` command is disabled until an AI key is configured. The default provider is Google Gemini. Anthropic and OpenAI-compatible chat APIs are also supported.

The question is trimmed and limited to 4,000 characters by default. The reply is limited to 1,024 output tokens by default. Requests time out after sixty seconds. The system instruction asks Salibot to answer clearly and in the language/style intended for WhatsApp.

For the simplest configuration, choose one provider and configure only that provider's key plus any necessary provider/model choice. Avoid setting several provider keys at the same time because the current key-selection order prefers Gemini, then the generic key, then Anthropic, then OpenAI.

Important: Questions are sent to an external AI provider. Do not send private messages, passwords, WhatsApp session data, or personal information unless the provider and your privacy policy permit it. AI answers may be incorrect and should not be treated as guaranteed facts.

8 Files, storage, and privacy

8.1 What is stored

File or folder	Purpose and sensitivity
`bots.json`	List of bot names. Needed to recreate the same bot entries after restart.
`allowed_<bot>.json`	Allowed groups, tracked groups, and all feature/permission choices for one bot. Group identifiers may be private.
`auth_info_<bot>/`	WhatsApp linked-device credentials. Highly sensitive: anyone who obtains usable credentials may be able to access the linked session.
`auth.json`	Website administrator names and scrypt password records. The original password is not stored, but the file is still sensitive.
`change.json`	Optional credential-change input. When valid content is found, it replaces the web user record and is then cleared. This is an advanced maintenance mechanism.
`deleted_cache/`	Temporary recovered media used by anti-delete and View Once features. May contain private images, videos, audio, stickers, or documents.
`logs/server.log`	Runtime events, errors, commands, connection details, and web request paths. It may contain private operational identifiers.
In-memory maps	Recent message copies, sessions, login-attempt counters, interactive command flows, connection states, and queued alerts. These disappear on restart.

8.2 Persistent Railway storage

Without a persistent volume, files inside a replaced Railway container may disappear. When a Railway volume is attached, Railway supplies `RAILWAY_VOLUME_MOUNT_PATH`; Salibot automatically uses it as the data directory. `APP_DATA_DIR` or `DATA_DIR` can override that behavior for advanced deployments.

The application code is packaged separately from runtime data. The Docker build excludes local passwords, linked-device folders, bot lists, logs, recovered media, tests, reports, and documentation from the image.

8.3 What to back up

For a recoverable service, back up the persistent volume containing:

- `bots.json`;
- every `allowed_<bot>.json`;
- every `auth_info_<bot>/` folder; and
- `auth.json`.

Backups of `deleted_cache/` and logs are optional and may increase privacy risk. Encrypt backups, restrict access, and define a deletion period. Never publish linked-device credentials or include them in support screenshots.

9 Website security in simple terms

Protection	What it means
No default administrator password	A fresh installation requires an explicit web username and password instead of silently using `admin/admin`.
Password hashing	The saved record uses scrypt with a random salt. Login recomputes the result instead of reading a plain password.
Timing-safe comparison	Secret comparisons are designed to reduce information leaks through response timing.
Turnstile	When both keys are configured, Cloudflare checks that the login visitor passed the challenge. The server validates the token; the visible widget alone is not trusted.
Login throttling	Public and authentication routes have request limits. Five failed credential/Turnstile attempts cause a fifteen-minute login lock for that IP.
Session cookie	The cookie is HTTP-only, SameSite=Lax, and Secure when the request uses HTTPS. It does not expose the token to normal page scripts.
CSRF tokens	State-changing forms include a secret token so another website cannot silently submit actions through a logged-in browser.
Security headers	The server sends CSP, HSTS, frame blocking, no-sniff, no-store, referrer, cross-origin, and permissions restrictions.
Small form bodies	Form submissions larger than 10 KB are rejected.
Escaping and validation	Names, errors, paths, and log text are escaped before display; bot identifiers and redirects are restricted.
Protected administration	Every dashboard, bot-management route, log page, and administration API requires a valid session.
Non-root container	The production process runs as the low-privilege `node` user rather than the container administrator.

The `/health` route is intentionally public so Railway can check the service. `robots.txt` asks search engines not to index the site, and the sitemap is empty. These are helpful signals, not access controls.

Important: The web password protects powerful controls. Use a unique, long password; keep Turnstile enabled; protect the Railway and GitHub accounts with multi-factor authentication; and never add a public unrestricted command shell to this site. A stolen web session or password would expose bot management and private logs.

10 Pre-deployment security gate

Every push to `main`, pull request, weekly scheduled run, or manual workflow starts the GitHub security job. All stages are attempted, then one final gate blocks the release if a required stage failed.

No.	Stage	Plain-language question
1	Secrets detection	Did a verified password, token, or secret enter Git history?
2	Code quality	Does the JavaScript parse correctly and follow the ESLint safety rules?
3	SAST	Does static analysis find forbidden dangerous code patterns?
4	SCA	Do production dependencies contain known high/critical vulnerabilities?
5	Infrastructure scan	Does the Docker/workflow configuration contain a high/critical mistake?

No.	Stage	Plain-language question
6	License check	Are dependency licenses classified and approved by project policy?
7	Container scan	Does the exact built image contain a fixable high/critical vulnerability?
8	SBOM	Was a full machine-readable component inventory created?
9	Provenance	Did GitHub create signed evidence connecting the build to its source revision?
10	Image signing	Was the candidate image bundle signed and immediately verified?
11	Policy as code	Does the repository meet mandatory project rules such as non-root runtime and health check?
12	DAST	Does OWASP ZAP find a blocking problem while examining the running candidate?
13	Integration tests	Do the real web components pass the expected security behaviors?

The workflow uploads evidence for thirty days. The most important file is `gate-summary.json`. Deployment is allowed only when the final step says `SECURITY GATE PASSED`. The separate *13-Stage Pre-Deployment Security Gate* PDF explains each stage in more technical detail.

11 Deployment on Railway

11.1 Container build

The Dockerfile uses Node.js 24.20.0 on Debian Bookworm Slim. A build stage installs production dependencies and downloads required media tooling. The final stage copies only the production application and dependencies, removes npm from the runtime image to eliminate unnecessary vulnerable tooling, creates the log folder, and switches to the non-root `node` user.

The container exposes port 8080. Railway normally supplies its own `PORT` value. A health check asks `/health` every thirty seconds after a short startup period.

11.2 Automatic deployment flow

1. A change is committed and pushed to GitHub.
2. GitHub builds and tests that revision.
3. If Railway **Wait for CI** is enabled, Railway waits for the GitHub status.
4. A red gate keeps the deployment blocked. The operator reads the failed stage and its report.
5. A green gate lets Railway build/start the new container.
6. Railway calls `/health`. If health succeeds, traffic moves to the new deployment.

11.3 Required Railway variables

For the intended production login with Cloudflare visible, configure exactly these four application secrets/settings:

```
WEB_ADMIN_USER=<your administrator name>
WEB_ADMIN_PASSWORD=<a long unique password>
TURNSTILE_SITE_KEY=<the public key for this website>
TURNSTILE_SECRET_KEY=<the private Cloudflare secret>
```

Railway supplies `PORT` automatically. If a volume is attached, Railway supplies the volume mount path automatically. The ordinary defaults for server name, server role, WhatsApp enabled state, and allowed hosts do not require custom variables.

The old Turnstile secret appeared in earlier Git history and should not be reused. Rotate it in Cloudflare and keep the replacement only in Railway's secret variables.

12 First-time setup checklist

1. Create or open the Railway service connected to the repository's `main` branch.
2. Attach a persistent volume before linking WhatsApp, so linked-device credentials and settings survive container replacement.
3. In Cloudflare Turnstile, create a widget for the Railway/custom hostname and obtain a new key pair.
4. Add the four production variables listed above in Railway.
5. Enable Railway **Wait for CI**.
6. Protect the GitHub `main` branch and require **All 13 DevSecOps stages**.
7. Push the deployment revision and wait for a green security gate.
8. Open the website, complete Turnstile, and sign in.
9. Open `bot1` or create a clearly named bot.
10. Link the WhatsApp account with QR or phone code.
11. Select allowed destination groups and tracked source groups.
12. Review command access, command locations, alert destination, and every feature switch.
13. Send `.ping`, then verify the website dashboard and log console.
14. Make an encrypted backup plan for the Railway volume.

13 Routine operation

13.1 Daily or frequent checks

- Confirm the website opens through HTTPS and Turnstile appears on login.
- Confirm expected bots say **Connected**.
- Read recent console errors without sharing screenshots that contain private identifiers.
- Check RAM/disk percentages and investigate repeated resource alerts.
- Confirm allowed and tracked groups still match operational needs.

13.2 Before changing code

- Back up persistent bot/session data.
- Test locally where possible.
- Push through a pull request or reviewed commit.
- Never bypass a red security gate merely to deploy faster.
- Confirm the green run belongs to the exact commit Railway will deploy.

13.3 After deployment

- Confirm Railway reports a healthy deployment.
- Sign in again because in-memory sessions reset.
- Check bot reconnection and re-link any logged-out account.
- Send a harmless `.ping` command.
- Check the console for startup or connection errors.

14 Troubleshooting

Symptom	What to check
Turnstile is missing	Confirm both Turnstile variables exist in the same Railway service and redeploy. One key alone disables the widget in the current code.
"Security check failed"	Confirm the site key belongs to the displayed hostname, the secret matches the same widget, and the token is not expired.
"Security check unavailable"	The server could not reach Cloudflare Siteverify or Cloudflare returned an unusable response. Check network and Railway logs.
Correct web password does not work	If `auth.json` already exists, changing Railway variables does not automatically replace that saved record. Use the supported credential-change procedure or restore the correct protected data.
Repeated login lock	Wait fifteen minutes and check that Railway/Cloudflare forwards the correct visitor IP. Five failures trigger the lock.
Bot remains disconnected	Open the bot page, request a fresh QR/code, check the log console, and verify the linked account did not log the device out.
No groups appear	The bot must be connected and already participate in those groups. Refresh after connection.
Alerts go nowhere	Check alert destination. If it is allowed groups, select at least one allowed group. Confirm the selected bot is connected.
Group changes are not reported	The group must be in tracked groups and the matching member/admin/group-setting switch must be enabled.
Deleted content is not recovered	Salibot must have received the original while online. Check the matching anti-delete switch and tracked-group requirement. Some encrypted or View Once media cannot be recovered.
`.u` does nothing	Only the owner may use it. Enable downloads, add an allowed group, use a supported URL, and check for private/expired/geo-blocked media or size limits.
`.ask` does nothing	Configure the chosen provider and key. Check model name, provider choice, quota, timeout, and console errors.
Speed test fails	Check outbound internet access and the configured speed-test URL. A provider/network failure can interrupt the stream.
Console is empty	Wait for application events, confirm `logs/server.log` is writable, and remember logs may reset when an ephemeral container is replaced.
GitHub gate is red	Open the failed numbered stage and download `security-reports-<commit>`. Read `gate-summary.json`, fix the cause, and push a new commit.
Railway deploys before tests finish	Enable **Wait for CI** and require the exact GitHub status check on the deployment branch.

Symptom	What to check
Data disappears after redeploy	Attach a Railway volume and verify the application is using the automatically supplied mount path. Restore an encrypted backup if available.
Restart loop	Read Railway runtime logs, check the first application error, verify required variables and volume permissions, and inspect health-check failures.

15 Advanced settings reference

Most operators should leave this section alone. A listed default means no Railway variable is required unless the behavior must be changed.

Setting	Default	Purpose
`PORT`	8080	Listening port; Railway normally supplies it automatically.
`SERVER_NAME`	Serveur Railway	Friendly server name used in alerts/status.
`SERVER_ROLE`	railway	Internal role label; current behavior is standalone.
`WHATSAPP_ENABLED`	true	Set false only for testing/scanning without WhatsApp connections.
`COMMAND_PREFIX`	period	Prefix before WhatsApp commands.
`APP_DATA_DIR` / `DATA_DIR`	automatic	Advanced override for runtime data location.
`RAILWAY_VOLUME_MOUNT_PATH`	supplied by Railway	Automatically selected when a Railway volume exists.
`LOG_FILE`	`logs/server.log`	Advanced override for the runtime log file.
`ALLOWED_HOSTS` / `WEB_ALLOWED_HOSTS`	unrestricted	Comma-separated hostnames accepted by the web server.
`ALLOW_LOCALHOST_HOSTS`	true	Adds localhost forms when an allowed-host list exists.
`PUBLIC_ROUTE_RATE_LIMIT_MAX`	60 per 15 min	Request limit for the public login page.
`AUTH_ROUTE_RATE_LIMIT_MAX`	20 per 15 min	Request limit for login submissions.
`BOT_AUTH_RATE_LIMIT_MAX`	12 per 15 min	Limit for QR, pairing, and disconnect operations.
`DELETED_CACHE_MAX_AGE_DAYS`	7	Age at which recovered-media files are removed.
`DELETED_CACHE_CLEANUP_INTERVAL_MS`	6 hours	How often old recovered-media files are checked.
`RESOURCE_ALERT_INTERVAL_MS`	5 minutes	How often RAM/disk estimates are checked.
`RESOURCE_ALERT_COOLDOWN_MS`	30 minutes	Minimum repeat interval while a resource stays high.
`RESOURCE_ALERT_RAM_PERCENT`	85	RAM alert threshold.
`RESOURCE_ALERT_DISK_PERCENT`	85	Application-disk alert threshold.
`PENDING_ALERT_TTL_MS`	1 hour	How long an undelivered server alert remains useful.
`SPEEDTEST_DOWNLOAD_URL`	Cloudflare 10 MB endpoint	Source used for the bandwidth test.
`SPEEDTEST_TIMEOUT_MS`	20 seconds	Network timeout for the speed test.
`SPEEDTEST_MAX_BYTES`	10 MB	Maximum data read by one speed test.
`AI_PROVIDER`	gemini	Selects Gemini, Anthropic, or OpenAI-compatible behavior.

Setting	Default	Purpose
`GEMINI_API_KEY` / `AI_API_KEY` / `ANTHROPIC_API_KEY` / `OPENAI_API_KEY`	none	Credential used by `.ask`. Configure only what is needed.
`AI_MODEL`	provider-dependent	Overrides the default model name.
`AI_BASE_URL`	provider endpoint	Advanced custom or compatible API address.
`AI_TIMEOUT_MS`	60 seconds	Maximum wait for an AI response.
`AI_MAX_TOKENS`	1,024	Maximum requested answer length.
`AI_MAX_QUESTION_CHARS`	4,000	Maximum question length sent to the provider.
`AI_SYSTEM_PROMPT`	Salibot instruction	Replaces the default assistant behavior instruction.
`ANTHROPIC_VERSION`	2023-06-01	Anthropic API-version header.
`COBALT_API_URL(S)`	none	Enables one or more optional Cobalt download endpoints.
`COBALT_API_KEY`	none	Optional credential for the configured Cobalt service.
`INSTAGRAM_COOKIES_FILE`	none	Cookie file for protected Instagram media; highly sensitive.
`INSTAGRAM_COOKIES_FROM_BROWSER`	none	Advanced yt-dlp browser-cookie source.
`YTDLP_USER_AGENT`	browser-like value	User agent used for social downloads.
`YTDLP_PROXY`	none	Optional proxy for yt-dlp traffic.
`YTDLP_FORMAT`	up-to-720p MP4 preference	Advanced yt-dlp format selection.
`YTDLP_MAX_FILESIZE`	80M	yt-dlp size limit.
`YTDLP_TIMEOUT_MS`	90 seconds	Maximum time for a yt-dlp operation.
`DOWNLOAD_REMOTE_BEFORE_SEND`	true	Downloads a remote Cobalt result locally before WhatsApp sending.
`REMOTE_DOWNLOAD_MAX_BYTES`	80 MB	Maximum accepted downloaded media size.

16 Known limitations and operational decisions

- WhatsApp behavior depends on an unofficial linked-device library and can change without notice.
- The server is currently standalone; labels referring to active/standby remain in parts of the code, but it does not contact a primary peer.
- Website sessions, message memory, flow state, and pending queues are not shared across instances and disappear on restart. Stored message text currently has no timed expiry, so long-running instances should also be watched for memory growth.
- Anti-delete and View Once recovery are best effort and cannot recover content never received by the running bot.
- The log file and deleted-media cache can grow between cleanup or redeployment; storage and privacy should be monitored.
- Contact-photo and group event accuracy depends on events delivered by WhatsApp/Baileys.
- Social downloads depend on third-party sites and may break when those sites change.
- AI output depends on an external service, cost/quota, model availability, and provider policy.
- The web console is read-only. A general-purpose browser shell is deliberately absent because it would create remote command execution capability.

- The public health response is for Railway monitoring and should not be treated as a secret administration endpoint.
- The web interface is generated directly by the Node server; there is no separate frontend build or database.

17 Glossary

Term	Simple meaning
API	A controlled way for one program to ask another program for data or an action.
Bot	One independent WhatsApp linked-device connection managed by Salibot.
Container	A packaged environment containing the application and the software it needs.
Cookie	A small browser value used here to remember an authenticated session.
CSRF	An attack where another website tries to make a logged-in browser submit an unwanted action. A private form token blocks it.
Dashboard	The protected administration pages visible after login.
Deployment	A specific version of the application running on Railway.
Environment variable	A setting supplied outside the source code, commonly used for passwords and provider configuration.
GitHub Actions	GitHub's automation system that runs this project's security checks.
Health check	A small public request Railway uses to decide whether the application is alive.
Log	A time-ordered record of what the application did or what error occurred.
Persistent volume	Storage that remains when Railway replaces or restarts a container.
Rate limit	A rule that stops one visitor from making too many requests in a short time.
SBOM	A list of software components inside the built application image.
Session	The server-side record that keeps an administrator signed in.
Turnstile	Cloudflare's human-verification system used before password validation.
Tracked group	A WhatsApp group watched as a source of events.
Allowed group	A WhatsApp group normally used as a destination for alerts or downloaded media.

18 Quick reference

18.1 The safest normal configuration

- Four Railway variables: web username, strong web password, Turnstile site key, Turnstile secret.
- One encrypted persistent Railway volume.
- Wait for CI enabled and the GitHub security gate required.
- Commands limited to the owner or group administrators unless broad access is intentional.
- Only necessary groups selected as allowed/tracked.
- Regular backups and a defined retention period for recovered media/logs.

18.2 Where to go

Need	Location
Sign in	Website root `/`
See all bots/resources	`/dashboard`
Read live application logs	`/logs` or **Logs serveur** in the sidebar
Manage one bot	Select **Manage** beside that bot
Choose destinations	Bot page → **Allowed groups**
Choose monitored groups	Bot page → **Tracked groups**
Connect WhatsApp	Bot page → QR code or phone number
See deployment/build logs	Railway dashboard, not the Salibot web console
See security result	GitHub Actions → **Pre-deployment security gate**

End of guide

This document describes the repository implementation as inspected on 10 September 2026.